From the parts bin to live analysis: connecting your AIRSAFE hardware

Plan for about 1 to 1.5 hours the first time (the first Docker build alone can take 10 to 25 minutes). Do the stages in order: every stage proves one thing, so when something fails you know exactly where.

Honest status. The Rust server and the firmware have never been compiled by me (no toolchain in my environment). The commissioning tool, analysis, ledger tools and web app were tested against a reference server. If the Docker build or the firmware build stops with an error, copy the error to me; see "What to send me" at the end.

---

0. What you need

Hardware

- ESP32 DevKit (ESP32-WROOM-32) and a USB cable that carries data (many charging cables do not)
- Sensors from your diagram: MH-Z19B (CO2), PMS5003 (PM), SHT31 (temperature/humidity), SGP30 (TVOC), HLK-LD2410 (presence)
- SSD1306 OLED (I2C), microSD module with a FAT32 card (32 GB or less), RGB LED with three 220 ohm resistors
- A solid 5 V supply (2 A) or a powered USB hub, breadboard and jumper wires, a multimeter
- Optional and **off by default**: XGZP6847A pressure sensor, analog air-velocity sensor (see section 9)

On the PC

- Docker Desktop (Windows: with WSL2 enabled), Node.js 22 or newer, VS Code with the PlatformIO extension
- USB driver for your board's USB chip if Windows does not see it (CP210x or CH340)
- The PC and the ESP32 on the **same 2.4 GHz WiFi** (the ESP32 cannot use 5 GHz). Know the WiFi name and password.

---

1. Start the software (10 to 25 min)

```
.\scripts\start-windows.ps1       # Windows (PowerShell, in the project folder)
./scripts/start.sh                # Linux / macOS
```

It builds and starts PostgreSQL, a local test chain and the API with the web app, waits until `http://localhost:8000/api/health` says `"ok":true`, prints your PC's LAN address and opens the browser.

1. In the web app choose **Register**, role *Master admin*, vertical *All*, and create your account (email + password).
2. You should see the dashboards with simulated data (tag SIMULATED). That is normal: the server simulates every room that has no real device. Your device will take over its room automatically.

Pass: the browser shows the overview and `http://localhost:8000/api/health` shows `{"ok":true,...}`.

---

2. Wire the hardware

Power off while wiring. **Common ground for everything.** Wire in groups and test each group (section 3) before the next.

Module	Module pin	ESP32 pin	Voltage	Notes
MH-Z19B CO2	Vin	5V	5 V	needs 4.5 to 5.5 V, about 150 mA peaks
	GND	GND		
	TX	**GPIO17**		sensor TX goes to ESP32 RX (UART2)
	RX	**GPIO16**		sensor RX goes to ESP32 TX
PMS5003 PM	VCC	5V	5 V	fan draws about 100 mA; fan must spin
	GND	GND		
	TX	**GPIO4**		UART1 RX
	RX	**GPIO2**		UART1 TX. GPIO2 is a boot pin: see Troubleshooting if flashing fails
	SET, RESET	leave open		
SHT31	VIN	3V3	3.3 V	I2C 0x44 (address pin low)
	GND, SDA, SCL	GND, **GPIO21**, **GPIO22**		

SGP30	VIN	3V3	3.3 V	I2C 0x58
	GND, SDA, SCL	GND, **GPIO21**, **GPIO22**		same bus as the others
SSD1306 OLED	VCC	3V3	3.3 V	I2C 0x3C
	GND, SDA, SCL	GND, **GPIO21**, **GPIO22**		
HLK-LD2410 presence	VCC	5V	5 V	about 80 mA
	GND	GND		
	TX	**GPIO27**		read through software serial
	RX	**GPIO26**		
microSD (SPI)	CS	**GPIO5**	module-dependent	most modules take 5 V and level-shift for you
	SCK, MISO, MOSI	**GPIO18**, **GPIO19**, **GPIO23**		
	VCC, GND	5V (or 3V3 per module), GND		
RGB LED (common cathode)	R, G, B	**GPIO25**, **GPIO33**, **GPIO32**		each through a 220 ohm resistor; common leg to GND

Rules that prevent most problems

- **TX goes to RX, RX goes to TX** (crossed). If a UART sensor is silent, swap the two wires first.
- The I2C modules all share GPIO21/GPIO22; each has its own address, so just connect them in parallel.
- **Power:** the ESP32 plus PMS5003, MH-Z19B and LD2410 can draw about 600 to 700 mA. A laptop USB port may brown out (random resets). Use a 2 A charger or a powered hub. While **flashing**, power the 5 V sensors from the supply, not the PC.
- Never feed 5 V into a 3V3 pin. Never connect two different supplies to the same rail without a diode.
- Before applying power, with the multimeter check that 5V and GND are **not** shorted.
- Keep the SHT31 several centimetres away from the ESP32 (its heat adds 1 to 3 degrees to the temperature).

---

3. Test each part alone (bring-up sketches)

Open a folder in `firmware/bringup/` in VS Code, press Upload (PlatformIO), then Serial Monitor (115200).

1. **`i2c_scan`**: expected `0x3C`, `0x44`, `0x58`. Nothing found means SDA/SCL, 3.3 V or GND. (SHT31 at `0x45` means its address pin is high: change `sht31.begin(0x44)` to `0x45` in the firmware.)
2. **`uart_probe`**: prints one line per sensor every few seconds:

   ```
   CO2  MH-Z19B : OK  612 ppm            (400 or 0 in the first 3 minutes after power-on is normal)
   PM   PMS5003 : OK  PM1.0=4 PM2.5=7 PM10=9 ug/m3
   Occ  LD2410  : OK  data frames at 256000 baud  -> run ld2410_set_baud to change it to 9600
   ```

3. **`ld2410_set_baud`** (once): the LD2410 ships at 256000 baud but the firmware reads it at 9600. Run this sketch; it finds the current baud, switches the sensor to 9600, restarts it and confirms with `SUCCESS`. The setting is stored in the sensor. (Run `uart_probe` again: the LD2410 line must now say 9600.)

Pass: every sensor you wired answers OK. A sensor that stays silent here will be silent in the main firmware too.

---

4. Connect the node to the software

4.1 Let the ESP32 reach your PC (Windows, once). Open PowerShell **as Administrator**:

```
.\scripts\open-firewall.ps1
```

Then, from a **phone on the same WiFi**, open `http://<your PC's LAN IP>:8000/api/health` (the start script printed the IP). You must see `{"ok":true,...}`. If not, the ESP32 will not reach it either: check the network is "Private", not "Public", and that the router has no "client isolation" (guest networks usually do).

4.2 Provision and configure in one command (needs Node 22; run in the project folder):

```
node tools/connect_hardware.mjs setup --email you@example.com --password YOURPASSWORD --room iso1 \
     --wifi-ssid "YourWiFi" --wifi-pass "YourWiFiPassword" --sign
```

It checks the API, creates the device in room `iso1` (change `--room` to any room id on the Devices page), finds your PC's LAN address, and

writes **`firmware/airsafe_node/config_local.h`** with WiFi, API address, device id, API key and (because of `--sign`) a fresh Ed25519 identity for the device. You edit nothing by hand. Re-running with the same `--device-id` is refused (the key is shown only once): pick a new id.

4.3 Flash the real firmware (not the simulation build):

```
cd wokwi
pio run -e esp32_real -t upload       # or PlatformIO sidebar: esp32_real -> Upload
pio device monitor -b 115200          # serial monitor
```

4.4 Read the self-test in the serial monitor right after boot. Every line should say OK:

```
========== AIRSAFE SELF-TEST ==========
CO2  MH-Z19B  (UART2 RX=GPIO17 TX=GPIO16): OK
PM   PMS5003  (UART1 RX=GPIO4 TX=GPIO2): OK
T/RH SHT31    (I2C 0x44): OK
TVOC SGP30    (I2C 0x58): OK
Occupancy LD2410 (soft-serial RX=GPIO27 TX=GPIO26 at 9600 baud): OK
XGZP6847A pressure: off (unverified, see config.h) | Air velocity: off (needs calibration, see config.h)
OLED SSD1306 (I2C 0x3C): OK | microSD: OK
WiFi: connected  IP 192.168.1.77  signal -58 dBm
API  http://192.168.1.50:8000: reachable
=======================================
```

Each failing line says what to check. A sensor that fails does **not** stop the node: it still publishes everything else (only CO2 is essential), and the web app shows exactly what is missing.

Pass: WiFi connected, API reachable, CO2 OK.

---

5. See the analysis

The node warms up for **3 minutes** (the MH-Z19B needs it; the serial monitor and OLED show a countdown), then publishes every 30 seconds. Watch it arrive:

```
node tools/connect_hardware.mjs watch --email you@example.com --password YOURPASSWORD --room iso1
```

The first hardware reading prints `Hardware is reporting`, then a rolling analysis. In the web app, open the room's dashboard (Hospital -> Isolation 1): the card is tagged **HARDWARE** (and **SIGNED**), the header says **LIVE · 1/22 ON HARDWARE**, and the room's detail panel has the **Analysis** section: CO2 and PM2.5 statistics and minutes above thresholds, time in each risk band, sensor-health warnings and plain-language insights, for 15 min / 1 h / 6 h / 24 h.

How to read the ventilation number. AIRSAFE estimates air changes per hour (ACH) from how fast CO2 falls after people leave: it needs at least 10 minutes of an *empty* room (occupancy 0) following a period where CO2 had risen 150 ppm or more. To see it: have people in the room for 20 to 30 minutes, then leave and close the door; after 15 minutes the panel reports an estimate. It is an estimate (single-zone, well-mixed assumption): compare it with your facility's specification, do not treat it as a certified measurement. The LD2410 reports **presence** (0 or 1), not a head count.

Pass: the room shows HARDWARE, CO2 matches the OLED, and the Analysis section fills in.

---

6. Verify the data cannot be altered (optional, takes 2 minutes)

1. Web app -> **Ledger** -> **Seal now** (otherwise it seals every minute with the Docker setup).
2. Open the room -> **Verify this reading**: the browser recomputes the hash, checks the device signature, the Merkle proof and the server's checkpoint signature.
3. `node tools/verify_proof.mjs --email ... --password ... --reading <id> --rpc http://localhost:8545` also asks the (local test) chain.

---

7. Placement and calibration (this decides whether the numbers are meaningful)

- **MH-Z19B:** mount at breathing height, away from doors, windows, vents and your own breath. Its automatic calibration assumes it sees fresh air (about 400 ppm) at least once a day for 20 minutes: in a permanently occupied room, calibrate manually outdoors once (hold in fresh air 20 min, then use the sensor's zero-point command) or expect a slow drift.
- **PMS5003:** the inlet and outlet must be free (do not box it in); keep away from kitchens and steam during tests.
- **SGP30:** learns its baseline over about 12 hours; early TVOC values are rough. The first 15 seconds always read 0 ppb.
- **LD2410:** aim it at the room; it detects still people by their breathing. Metal and thick walls attenuate it.

Plausibility check of the first readings:

Quantity	Expected
CO2	400 to 450 ppm outdoors, 420 to 600 empty ventilated room, 600 to 1200 occupied, above 1500 poor
PM2.5	2 to 12 µg/m³ clean indoor air; cooking, incense or candles push it above 50
Temperature / humidity	about 18 to 30 °C, 30 to 70 % RH
TVOC (SGP30)	0 ppb for 15 s, then typically 20 to 200 ppb

8. Troubleshooting

Run the doctor first; it checks the chain and tells you the fix:

```
node tools/connect_hardware.mjs doctor --email you@example.com --password YOURPASSWORD --room iso1 --device-id esp32-iso1-01
```

Symptom	Likely cause	Fix
`docker compose` / build error	Rust code has not been compiled before	send me the error text (see below)
`/api/health` not ok	database still starting, or API crashed	`docker compose logs db api`
PlatformIO does not see the board	charge-only cable, missing driver	try another cable; install CP210x/CH340 driver
Upload says "Failed to connect"	boot pins held by wiring	hold the BOOT button during upload; **disconnect the PMS5003 RX wire (GPIO2) while flashing**, reconnect after
Random resets / brown-out messages	power too weak	2 A supply or powered hub; flash with 5 V sensors unpowered
WiFi NOT CONNECTED	5 GHz network, wrong password	use the 2.4 GHz SSID; re-run `setup` with the right `--wifi-pass`
API UNREACHABLE (status -1)	wrong IP, firewall, router isolation	use the PC's LAN IP, run `open-firewall.ps1`, test from a phone, avoid guest WiFi; re-run `setup --api-host <ip>` if the PC's IP changed
`HTTP 401` in the serial monitor	device id/key mismatch, or signing key does not match	re-run `setup` with a new `--device-id` and reflash
`Reading rejected` (422)	a value outside the accepted range (faulty sensor)	the reading is dropped on purpose; check that sensor with `uart_probe`
CO2 NO ANSWER	TX/RX not crossed, no 5 V	sensor TX to GPIO17, sensor RX to GPIO16, measure 5 V at Vin
PM NO ANSWER	fan not spinning, TX/RX	5 V to VCC, sensor TX to GPIO4
SHT31 / SGP30 NOT FOUND	I2C wiring, address	`i2c_scan`; SHT31 at 0x45 needs `sht31.begin(0x45)`
LD2410 NO FRAMES	baud not 9600	run `ld2410_set_baud` once, then power-cycle
Nothing for 3 minutes	normal warm-up	wait; set `#define WARMUP_MS 0` in `config_local.h` only for bench tests (CO2 will be inaccurate)
Card says STALE	no report for 2 minutes	power, WiFi range, PC IP change; run `doctor`
CO2 stuck at one value	sensor warming up or faulty	the analysis flags it; power-cycle, check wiring
Other rooms still show SIMULATED	by design	set `SIM_ENABLED: "0"` for `api` in `docker-compose.yml` to switch the simulator off
"Ventilation not measurable"	no empty-room decay yet	see section 5

9. Known limitations

- **XGZP6847A pressure and the analog air-velocity sensor are OFF by default** (`ENABLE_DIFF_PRESSURE`, `ENABLE_AIR_VELOCITY` in `config.h`). Their conversion code is unverified (guessed register layout, placeholder voltage curve); a wrong pressure would also distort the isolation-room risk score. Enable them only after checking against your datasheet and a known reference.
- The relay output and the TP4056/battery power section are not part of this software (see the circuit review in the guide PDF).
- Occupancy is presence (0 or 1), not a count. Anchoring on a public chain needs your own funded key and contract; the Docker setup uses a local test chain.

What to send me if something does not work

1. The text of the failing command (copy from the terminal).
2. The **SELF-TEST block** from the serial monitor.
3. The output of the `doctor` command.
4. For server problems: `docker compose logs api --tail 80`.